
C: Como Programar

6ª Edição · Paul Deitel & Harvey Deitel

Capítulo 7

Ponteiros em C

Exercícios de Autorrevisão (7.1–7.6)

Resolvidos passo a passo, abordagem incremental

Capítulos 1 + 2 + 3 + 4 + 5 + 6 + 7

Construindo sobre os Capítulos Anteriores

No Capítulo 7, abordamos um dos tópicos mais poderosos – e desafiadores – de C: os **ponteiros**. Construindo sobre os fundamentos dos Capítulos 1–6, incorporamos:

- **Ponteiros** – variáveis que armazenam endereços de memória
- **Operador de endereço &** – obtém o endereço de uma variável
- **Operador de desreferência *** – acessa o valor apontado
- **Chamada por referência simulada** – passar ponteiros para funções
- **Aritmética de ponteiros** – somar/subtrair inteiros a ponteiros
- **Equivalência entre arrays e ponteiros** – nomes de arrays como ponteiros constantes
- **Ponteiros e const** – 4 combinações de privilégios
- **Ponteiros para funções** – passar e armazenar comportamento
- **Strings como arrays de char** – preparação para o Cap. 8

Boa prática de programação

Ponteiros são a chave para entender a memória em C. Domine os operadores **&** (endereço) e ***** (desreferência), a equivalência array/ponteiro e a aritmética de ponteiros. Esses três tópicos resolvem 90% das dificuldades com ponteiros.

Contents

1	Exercício 7.1 – Preenchimento de Lacunas	2
2	Exercício 7.2 – Verdadeiro ou Falso	4
3	Exercício 7.3 – Array numbers	6
4	Exercício 7.4 – Variáveis number1, number2 e fPtr	8
5	Exercício 7.5 – Protótipos e Cabeçalhos	9
6	Exercício 7.6 – Identificar e Corrigir Erros	10

1. Exercício 7.1 – Preenchimento de Lacunas

Item (a)

Resposta detalhada

Resposta: Uma variável de ponteiro contém como valor o(a) **endereço** de outra variável.

Um ponteiro **não** contém um valor de dados (como 42 ou 3.14); ele contém o *endereço de memória* onde um valor está armazenado. Em C, declaramos ponteiros com asterisco antes do nome:

```
1  int *ptr;           /* ptr e um ponteiro para int */
2  int x = 42;
3  ptr = &x;           /* ptr agora contem o ENDERECO de x
   */
4  printf( "%d\n", *ptr ); /* *ptr desreferencia:
   imprime 42 */
```

Analogia: se uma variável é uma “caixa” com um valor dentro, um ponteiro é um “papel com o número da caixa” – não a caixa em si, mas o caminho para encontrá-la.

Item (b)

Resposta detalhada

Resposta: Os três valores para inicializar um ponteiro são **0**, **NULL** e **um endereço**.

- **0** – valor inteiro literal zero; tratado como ponteiro nulo em contexto de ponteiro
- **NULL** – constante simbólica definida em `<stddef.h>` e outros cabeçalhos; representa “nenhum endereço”
- **Um endereço** – de outra variável, obtido com `&`

Boa prática de programação

Use **NULL** (não 0) ao inicializar ponteiros, pois deixa a intenção explícita: `int *ptr = NULL;`. Inicializar ponteiros é fundamental – ponteiros não inicializados (“ponteiros perdidos”) causam comportamento imprevisível.

Item (c)

Resposta detalhada

Resposta: O único inteiro que pode ser atribuído a um ponteiro é **0**.

```
1  int *ptr;
2  ptr = 0;           /* OK: equivale a NULL */
```

```
3 ptr = NULL;      /* OK: forma preferida */  
4 ptr = 100;      /* ERRO (em compiladores modernos) */
```

A regra existe para evitar atribuições acidentais de números “mágicos” a ponteiros – algo que quase certamente causaria falha de segmentação. O zero é a exceção por ter um significado especial como “endereço inválido”.

2. Exercício 7.2 – Verdadeiro ou Falso

Item (a) – & aplica-se a constantes, expressões e registradores

FALSO

O operador de endereço & aplica-se **somente a variáveis**. Não pode ser aplicado a:

- **Constantes:** &5 é inválido (constantes literais não têm endereço fixo).
- **Expressões:** &(x + y) é inválido (expressões temporárias).
- **Variáveis register:** elas podem estar em registradores da CPU, que não possuem endereço na RAM.

Item (b) – Ponteiro void pode ser desreferenciado

FALSO

Um ponteiro `void *` **não** pode ser desreferenciado diretamente, porque o compilador não sabe *quantos bytes* ler nem *como interpretá-los*.

Para desreferenciar, primeiro deve-se converter (*cast*) o ponteiro para o tipo apropriado:

```
1 void *vp;  
2 int x = 42;  
3 vp = &x;  
4  
5 /* *vp; ERRO -- nao se pode desreferenciar void* */  
6 int valor = *( ( int * )vp ); /* OK: converte para  
   int* primeiro */
```

*Por que existe void *?* É o “ponteiro genérico”: pode receber endereço de qualquer tipo, útil em funções de uso geral (como `malloc`, que estudaremos no Cap. 12).

Item (c) – Ponteiros de tipos diferentes podem ser atribuídos sem cast

FALSO

Ponteiros de tipos diferentes **não** podem ser atribuídos uns aos outros sem cast (com a única exceção do `void *`). Por exemplo:

```
1  int    *ip;
2  float  *fp;
3  ip = fp;           /* ERRO ou aviso de compilacao */
4  ip = ( int * )fp;  /* OK com cast (mas perigoso!) */
```

A regra existe porque a aritmética de ponteiros e a desreferenciação dependem do tipo (`int *` avança 4 bytes; `float *` também 4 bytes; mas `double *` avança 8 bytes, etc.). Atribuir entre tipos incompatíveis pode levar a interpretações incorretas dos dados.

Exceção: `void *` pode ser atribuído a/de qualquer outro tipo de ponteiro sem cast explícito.

3. Exercício 7.3 – Array numbers

Resposta detalhada

a) Definir e inicializar:

```
1 #define SIZE 10
2 float numbers[ SIZE ] = { 0.0, 1.1, 2.2, 3.3, 4.4,
3                           5.5, 6.6, 7.7, 8.8, 9.9
                           };
```

b) Ponteiro para float:

```
1 float *nPtr;
```

c) Imprimir com notação de subscrito:

```
1 for ( i = 0; i < SIZE; ++i ) {
2     printf( "%.1f ", numbers[ i ] );
3 }
```

d) Atribuir endereço inicial a nPtr (duas formas):

```
1 nPtr = numbers;           /* nome do array e o
                             endereço do 1o elemento */
2 nPtr = &numbers[ 0 ];     /* equivalente, mas
                             explícito */
```

e) Imprimir usando nPtr com notação ponteiro/deslocamento:

```
1 for ( i = 0; i < SIZE; ++i ) {
2     printf( "%.1f ", *( nPtr + i ) );
3 }
```

Note: `*(nPtr + i)` é equivalente a `nPtr[i]` ou `numbers[i]`.

f) Imprimir usando o nome do array como ponteiro:

```
1 for ( i = 0; i < SIZE; ++i ) {
2     printf( "%.1f ", *( numbers + i ) );
3 }
```

g) Imprimir subscritando nPtr:

```
1 for ( i = 0; i < SIZE; ++i ) {
2     printf( "%.1f ", nPtr[ i ] );
3 }
```

h) Referenciar o elemento 4 de quatro formas:

```
1 numbers[ 4 ]           /* notacao de subscrito de
                           array */
2 *( numbers + 4 )        /* ponteiro/deslocamento com
                           nome do array */
3 nPtr[ 4 ]               /* notacao de subscrito de
                           ponteiro */
4 *( nPtr + 4 )           /* ponteiro/deslocamento com
                           nPtr */
```

i) Endereço de nPtr + 8:

Endereço inicial: 1002500. Cada float ocupa 4 bytes. Aritmética:

$$1002500 + 8 \times 4 = 1002500 + 32 = \mathbf{1002532}$$

Valor armazenado em `numbers[8]`: **8.8**.

j) Endereço após `nPtr -= 4`, supondo `nPtr` aponta para `numbers[5]`:

- Endereço de `numbers[5]`: $1002500 + 5 \times 4 = 1002520$
- Após `nPtr -= 4`: $1002520 - 4 \times 4 = \mathbf{1002504}$

Esse endereço é `numbers[1]`, cujo valor é **1.1**.

Boa prática de programação

As **quatro notações equivalentes** – `numbers[i]`, `nPtr[i]`, `*(numbers + i)`, `*(nPtr + i)` – são uma das ideias mais poderosas de C. Domine-as e ponteiros deixam de parecer “mágicos”.

4. Exercício 7.4 – Variáveis number1, number2 e fPtr

Resposta detalhada

Considere `float number1 = 7.3;` e `float number2;`.

a) Definir ponteiro `fPtr`:

```
1 float *fPtr;
```

b) Atribuir endereço de `number1`:

```
1 fPtr = &number1;
```

c) Imprimir valor apontado:

```
1 printf( "0 valor de *fPtr e %f\n", *fPtr );
```

d) Atribuir o valor apontado a `number2`:

```
1 number2 = *fPtr;
```

e) Imprimir `number2`:

```
1 printf( "0 valor de number2 e %f\n", number2 );
```

f) Imprimir endereço de `number1` com `%p`:

```
1 printf( "0 endereco de number1 e %p\n", &number1 );
```

g) Imprimir endereço armazenado em `fPtr`:

```
1 printf( "0 endereco armazenado em fPtr e %p\n",  
        fPtr );
```

O valor impresso é o mesmo que `&number1`? Sim! Como `fPtr = &number1` no item (b), `fPtr` contém exatamente o endereço de `number1`.

Os dois `printf` produzirão a mesma saída.

Nota sobre `%p`: é o especificador padrão para imprimir endereços em formato hexadecimal. A representação exata depende do compilador (geralmente `0x7fffe1a23bdc` ou similar).

5. Exercício 7.5 – Protótipos e Cabeçalhos

Resposta detalhada

a) **Cabeçalho de exchange:** dois ponteiros para float, retorna void.

```
1 void exchange( float *x, float *y )
```

b) **Protótipo de exchange:** adicione ;.

```
1 void exchange( float *x, float *y );
```

c) **Cabeçalho de evaluate:** recebe int x e um ponteiro para a função poly (que recebe int e retorna int); retorna int.

```
1 int evaluate( int x, int (*poly)( int ) )
```

d) **Protótipo de evaluate:**

```
1 int evaluate( int x, int (*poly)( int ) );
```

Sintaxe importante: a expressão `int (*poly)(int)` é lida como “poly é um ponteiro para função que recebe um int e retorna um int”. Os parênteses ao redor de `*poly` são **essenciais**: sem eles, `int *poly(int)` seria interpretado como “função que retorna int *” – algo totalmente diferente!

Boa prática de programação

Ponteiros para funções são poderosos porque permitem *passar comportamento* como argumento – a base de *callbacks*, *strategy pattern* e programação funcional. Já vimos um exemplo prático: `qsort` aceita um ponteiro para função de comparação.

6. Exercício 7.6 – Identificar e Corrigir Erros

Considere as declarações:

```
1  int *zPtr;                                /* nao inicializado!
    */
2  int *aPtr = NULL;
3  void *sPtr = NULL;
4  int number, i;
5  int z[ 5 ] = { 1, 2, 3, 4, 5 };
6  sPtr = z;
```

Item (a)

Resposta detalhada

Código com erro: ++zptr;

Erros:

- a) zptr (minúsculo) é diferente de zPtr; C é *case-sensitive*.
- b) Mesmo escrevendo ++zPtr, há erro: zPtr nunca foi inicializado. Incrementar um ponteiro indefinido leva a comportamento indefinido.

Correção:

```
1  zPtr = z;                                /* inicializa antes de usar */
2  ++zPtr;                                  /* agora aponta para z[1] */
```

Erro comum de programação

Usar um ponteiro **não inicializado** é uma das principais causas de falhas em C. Sempre inicie ponteiros (com NULL, 0 ou um endereço válido) antes de usá-los.

Item (b)

Resposta detalhada

Código com erro:

```
1  /* usa o ponteiro para obter o primeiro valor do array
    */
2  number = zPtr;
```

Erro: zPtr é um *endereço* (ponteiro), não um valor inteiro. Atribuí-lo a number (que é int) gera aviso ou erro.

Correção: desreferenciar com *:

```
1  number = *zPtr; /* obtém o valor apontado */
```

Item (c)

Resposta detalhada

Código com erro:

```
1 number = *zPtr[ 2 ];
```

Erro: `zPtr[2]` já é um *valor int* (não um ponteiro). Aplicar `*` a um inteiro é incorreto.

Correção:

```
1 number = zPtr[ 2 ]; /* equivale a *(zPtr + 2) */
```

Item (d)

Resposta detalhada

Código com erro:

```
1 for ( i = 0; i <= 5; i++ ) {  
2     printf( "%d ", zPtr[ i ] );  
3 }
```

Erro: `i <= 5` acessa `zPtr[5]`, que está **fora dos limites** do array `z` (índices válidos: 0 a 4).

Correção:

```
1 for ( i = 0; i < 5; ++i ) { /* < em vez de <= */  
2     printf( "%d ", zPtr[ i ] );  
3 }
```

Item (e)

Resposta detalhada

Código com erro: `number = *sPtr;`

Erro: `sPtr` é `void *` – não pode ser desreferenciado diretamente sem cast.

Correção:

```
1 number = *( ( int * )sPtr ); /* cast para int* antes  
    */
```

Item (f)

Resposta detalhada

Código com erro: `++z;`

Erro: O nome de um array (`z`) é um *ponteiro constante* – seu valor não pode ser modificado.

Correção: usar uma variável de ponteiro auxiliar:

```
1  int *p = z;  
2  ++p;           /* agora p aponta para z[1]; z  
                  permanece valido */
```

Alternativamente, usar subscrito direto: `z[1]` para acessar o segundo elemento.

Gabarito Rápido – Capítulo 7: Autorrevisão

Respostas Resumidas

7.1:

a) endereço b) 0, NULL, um endereço c) 0

7.2 – Verdadeiro/Falso (todos falsos):

- a) **Falso** – & só em variáveis (não register)
- b) **Falso** – void * não pode ser desreferenciado
- c) **Falso** – precisa de cast (exceto void *)

7.3: arrays e ponteiros; quatro notações equivalentes; endereços calculados pela aritmética de ponteiros (deslocamento × tamanho do tipo).

7.4: fPtr = &number1, desreferência com *fPtr, %p para endereços (idênticos para &number1 e fPtr).

7.5: sintaxe de ponteiros para funções: int (*poly)(int).

7.6: 6 erros corrigidos: ponteiro não inicializado, desreferência faltando, desreferência incorreta, <= em vez de <, falta cast em void *, e modificação de nome de array.